Assignment 2 based on CNN

Gaurav Sharma
Entry No: 2025JRB2022

Part I

Bird Image Classification

1 Introduction

The goal of this task was to build a deep learning model that classifies bird images into multiple categories. The dataset has about 12,000 images, which are spread across 15 classes.

Instead of using a large pre-trained architecture, the objective was to design and test custom convolutional neural network architectures and see how different design choices impact model performance.

Several aspects of the pipeline were explored:

- Network architecture depth
- Convolutional block design
- Learning rate selection
- Batch size
- Regularization techniques

The final model was selected based on validation accuracy and F1 score.

2 Data Preprocessing and Augmentation

All images were resized to 224×224 before they went into the network. To improve the model's ability to generalize and reduce overfitting, several data augmentation techniques were used during training.

The augmentations included:

- Random Resized Cropping
- Horizontal and Vertical Flips
- Small Random Rotations
- Color Jitter (brightness, contrast, saturation)

For validation and testing, only resizing and normalization were applied to ensure a consistent evaluation setting.

Images were normalized using ImageNet statistics:

$$\text{mean} = [0.485, 0.456, 0.406], \quad \text{std} = [0.229, 0.224, 0.225]$$

3 Architecture Exploration

The model architecture was developed iteratively through a few experiments.

- **Model 1:** The first model had three convolutional layers set up in residual blocks. Each block included two convolution layers, followed by batch normalization and ReLU activation. Residual skip connections helped improve gradient flow during training.
- **Model 2:** The second model was an improved version of the first one. It had 4 convolutional layers instead of 3. The rest of the architecture is the same. It turns out to be the best model architecture amongst the all that I could think of.
- **Model 3:** In the third model, I attempt to make this model a little deeper by introducing an additional convolutional layer in the block:
  - 1×1 convolution (channel mixing)
  - 3×3 convolution (spatial feature extraction)
  - 1×1 convolution (feature projection)

  It had three convolutional layers and the rest of the architecture is same as the first one.
- **Model 4:** Similar to what I did in Model 2, here I increased the depth layer 3 to 4. Turns out Model 3 and Model 4 give better results in initial epochs but in the long term their accuracy starts fading.

3.1 Architecture Comparison

To evaluate the effectiveness of the models explored during experimentation, Figure 1 illustrates a comparative analysis of their performance across key metrics.

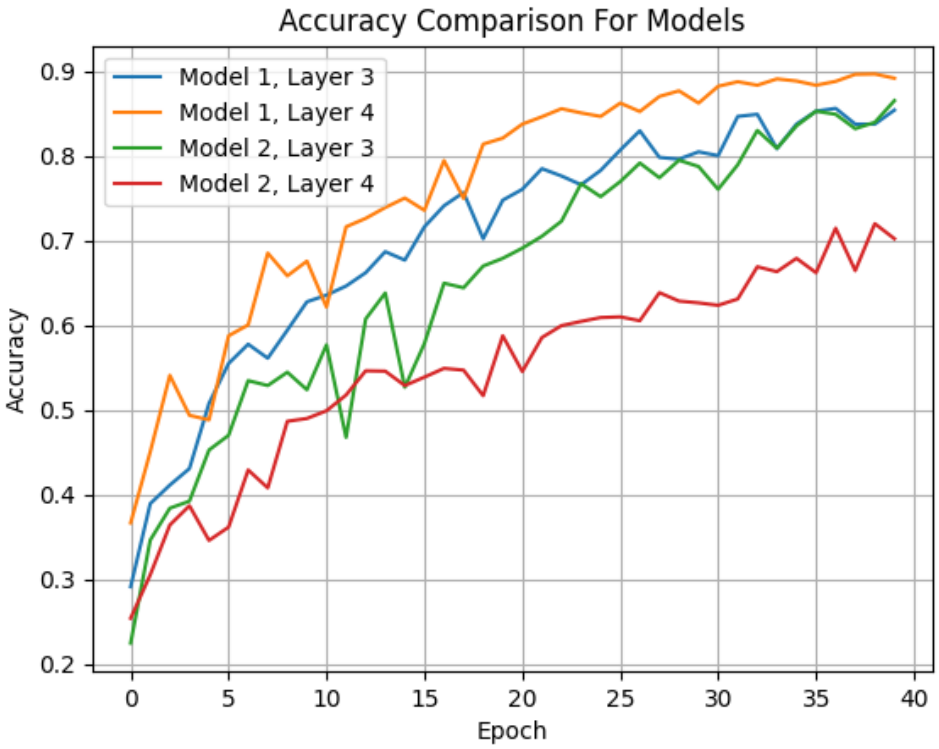


Figure 1: Performance comparison of the different model architectures tested during experimentation.

4 Hyperparameter Experiments

After identifying the best architecture, the next step was to tune important training hyperparameters.

4.1 Learning Rate

Three learning rates were evaluated:

- 0.005
- 1×10^{-3}
- 1×10^{-4}

The higher learning rate (0.005) caused unstable training and slower convergence. On the other hand, the smallest learning rate (10^{-4}) resulted in smooth learning in a few epochs to achieve reasonable accuracy.

I was expecting for (10^{-4}) learning rate to be optimum but it tuns out that the learning rate 1×10^{-4} provided the best balance between stability and convergence speed.

Note: Along with that i also used cosine annealing, which is scheduling method which reduces the learning rate smoothly. This helps models converge more effectively by making large updates early and smaller, fine-tuned updates later.

4.2 Batch Size

Three batch sizes were tested:

- 16
- 32
- 64

Smaller batch sizes introduced higher gradient noise, which sometimes helped generalization but slowed down training. Larger batch sizes improved computational efficiency but did not significantly improve model accuracy.

The batch size of 32 produced the best validation performance and stable training behaviour.

4.3 Hyperparameter Comparison

Figure 2 and Figure 3 shows the comparison between the different hyperparameter configurations.

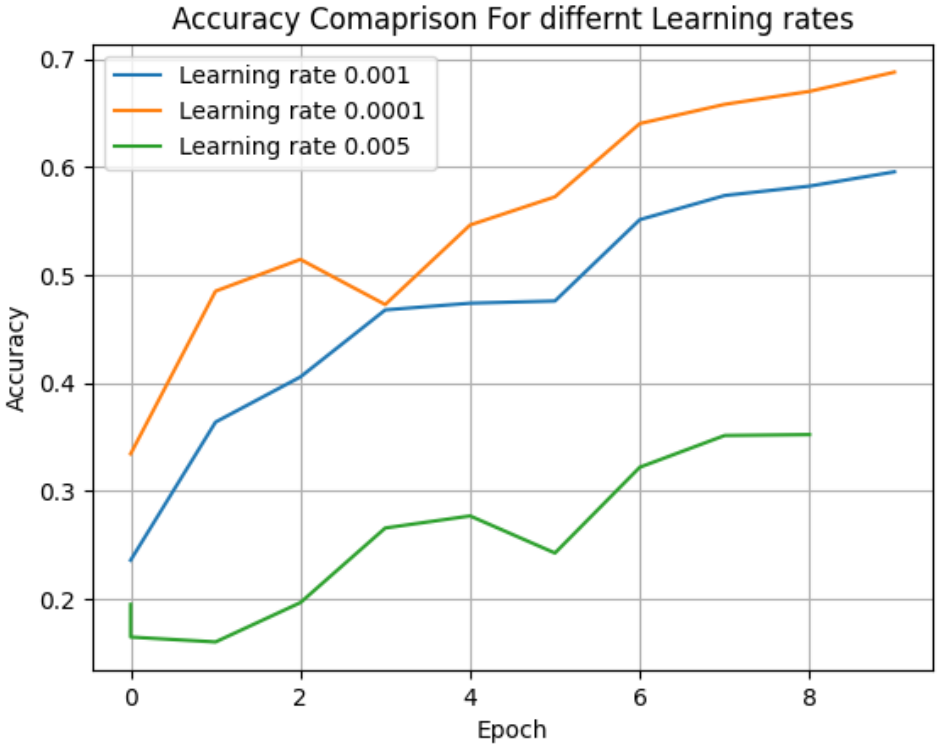


Figure 2: Comparison of learning rates

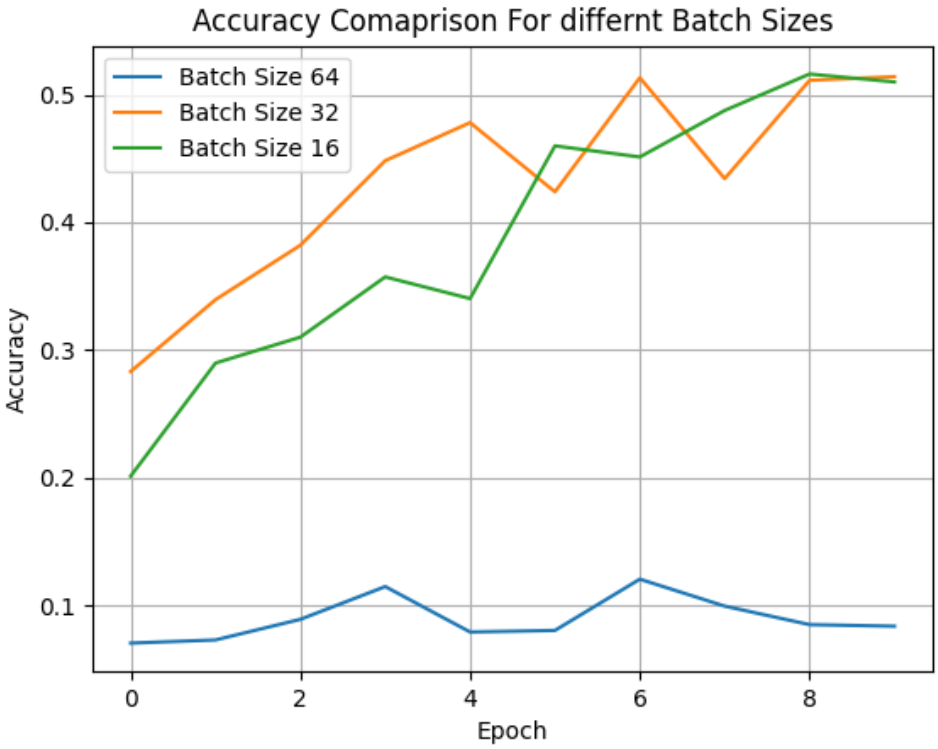


Figure 3: Comparison of batch sizes

5 Loss Function and Optimization

The model was trained using the cross entropy loss function, which is commonly used for multi-class classification tasks.

To address potential class imbalance in the dataset, class weights were incorporated into the loss function. These weights were calculated as the inverse frequency of each class.

Label smoothing with a value of 0.1 was also applied to prevent the model from becoming overly confident in its predictions.

The optimizer used for training was Adam with weight decay regularization. Adam was chosen due to its adaptive learning rate capabilities and generally strong performance across a wide range of deep learning tasks.

6 Regularization Techniques

Several regularization techniques were used to improve generalization:

- Data augmentation
- Dropout in the classifier layer(Based on Val accuracy)
- Weight decay in the optimizer
- Label smoothing in the loss function

- Early stopping based on validation accuracy

Additionally, a cosine annealing learning rate scheduler was used to gradually reduce the learning rate during training.

7 Result

The final model used the following configuration:

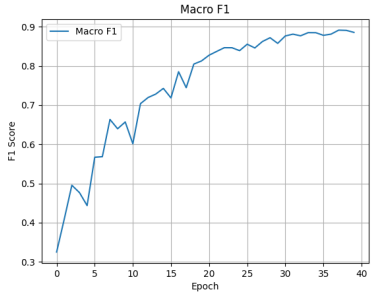
- Block:

$$x \xrightarrow{\text{conv1}} \text{bn1} \xrightarrow{\text{ReLU}} \text{out} \xrightarrow{\text{conv2}} \text{bn2} \longrightarrow \text{out} + \text{x} \xrightarrow{\text{ReLU}} \text{final out}$$

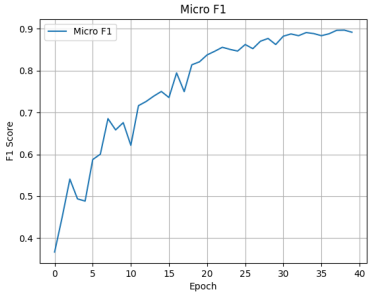
- Block: Residual CNN with 2 convolution layers per block
- Architecture: 4 Layer
- Input size: 224×224
- Optimizer: Adam
- Learning Rate: 1×10^{-4}
- Batch Size: 32
- Epochs: 40
- Loss Function: Cross Entropy with class weights and label smoothing
- Scheduler: Cosine Annealing
- Regularization: Dropout, data augmentation, weight decay

During training, multiple performance indicators were monitored to evaluate both learning stability and segmentation quality. These include the Macro F1 score, Micro F1 score, training accuracy, training loss, validation loss, and validation accuracy.

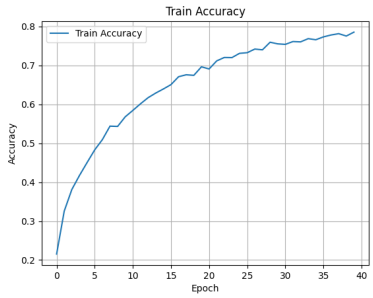
The following figures illustrate the training dynamics across epochs.



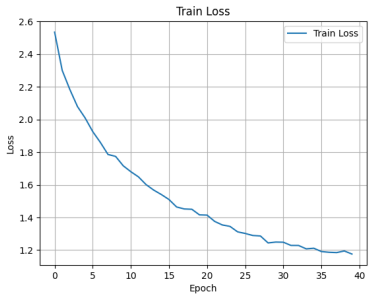
(a) Macro F1



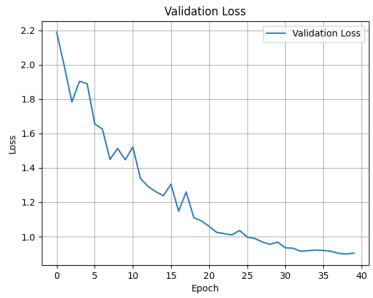
(b) Micro F1



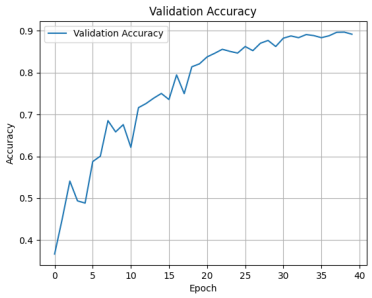
(c) Training Accuracy



(d) Training Loss



(e) Validation Loss



(f) Validation Accuracy

Figure 4: Training and validation metrics recorded during model optimization.

8 Conclusion

Through a series of architecture and hyperparameter experiments, it was observed that increasing model complexity does not always lead to better performance.

A slightly simpler architecture with improved convolutional blocks produced better results than a deeper model. Careful tuning of the learning rate and batch size also played a significant role in achieving stable and effective training.

I was not able to find the best solution, but I found a few optimum ones. Or maybe if it can run on more epochs than 40, it could reach upto 0.95-0.97 accuracy.

The final model achieved the best balance between accuracy, training stability, and computational efficiency.

Part II
Cell Image Segmentation

9 Cell Segmentation Strategy

9.1 Overview

The objective of this task is to perform **semantic segmentation** of cell images, where each pixel must be assigned to one of five classes: background or one of four different cell types. Unlike classification tasks, segmentation requires both spatial precision and contextual understanding of image structures.

To address this problem, a **U-Net based convolutional neural network** was used. U-Net is widely adopted in biomedical image segmentation because it effectively preserves spatial information while also capturing high-level semantic features through its encoder–decoder architecture with skip connections.

The final implementation uses a lightweight U-Net architecture trained using **cross-entropy loss** and optimized with the **Adam optimizer**. Model performance is evaluated using **Intersection over Union (IoU)** and **Dice score**, which are standard metrics for segmentation tasks.

9.2 Network Architecture

The segmentation network follows the classical **U-Net encoder–decoder design**. The architecture consists of three major components: an encoder, a bottleneck, and a decoder.

9.2.1 Encoder

The encoder progressively extracts hierarchical features from the input image using convolutional blocks followed by max-pooling operations. Each convolutional block contains two convolution layers, each followed by batch normalization and ReLU activation.

This design allows the model to capture increasingly abstract representations of cell structures as the spatial resolution decreases.

9.2.2 Bottleneck

The bottleneck layer represents the deepest part of the network and processes a compressed feature representation. This layer captures high-level contextual information before the decoder reconstructs the spatial details.

9.2.3 Decoder

The decoder progressively restores spatial resolution using transposed convolution layers. At each stage, feature maps from the encoder are concatenated with the upsampled features through **skip connections**. These connections help recover fine spatial details that are often lost during downsampling.

Finally, a 1×1 convolution layer produces the segmentation map with five output channels corresponding to the five classes.

9.3 Loss Function

The model is trained using **Cross-Entropy Loss**, which is widely used for multi-class classification problems.

In semantic segmentation, each pixel prediction is treated as a classification problem. Cross-entropy measures the difference between the predicted probability distribution and the true class label for each pixel.

$$L = -\sum_{c=1}^{C} y_c \log(\hat{y}_c) \tag{1}$$

where y_c represents the true class label and $\hat{y}_c$ represents the predicted probability for class c.

Cross-entropy was chosen because it:

- Handles multi-class pixel classification naturally
- Provides stable gradients during training
- Works well with softmax outputs

9.4 Evaluation Metrics

Two widely used segmentation metrics were used to evaluate model performance.

9.4.1 Intersection over Union (IoU)

Intersection over Union measures the overlap between the predicted segmentation and the ground truth mask.

$$IoU = \frac{\text{Intersection}}{\text{Union}} \tag{2}$$

A higher IoU indicates better alignment between the predicted and ground truth segmentation.

9.4.2 Dice Score

Dice score measures the similarity between predicted and ground truth masks.

$$Dice = \frac{2 \times \text{Intersection}}{\text{Prediction} + \text{GroundTruth}} \tag{3}$$

Dice is particularly useful in segmentation tasks because it is sensitive to small regions and imbalanced classes.

Both metrics were computed excluding the background class in order to focus on the segmentation quality of the cell types.

9.5 Optimizer

The network parameters were optimized using the **Adam optimizer**. Adam combines the advantages of momentum and adaptive learning rate techniques, making it well suited for deep neural networks.

Adam was selected because it:

- Converges faster than traditional stochastic gradient descent
- Automatically adapts learning rates for each parameter
- Requires minimal manual tuning

9.6 Hyper-parameter Tuning

The performance of deep learning models is highly sensitive to hyper-parameters such as the learning rate (LR) and batch size (BS). These parameters directly influence optimization stability, convergence speed, and the model's ability to generalize. Therefore, several combinations were experimentally evaluated to determine the most effective configuration.

Learning Rate. The learning rate controls the magnitude of parameter updates during gradient descent. If the learning rate is too large, the optimizer may overshoot minima in the loss landscape, causing unstable training. Conversely, an excessively small learning rate slows down convergence and increases training time.

In this study, three learning rates were evaluated: 1×10^{-4}, 1×10^{-3}, and 5×10^{-3}. Larger learning rates (10^{-3} and 5×10^{-3}) resulted in unstable optimization behavior, where the loss oscillated significantly and the validation score failed to improve consistently. This behavior occurs because large parameter updates prevent the optimizer from settling into stable minima.

In contrast, a smaller learning rate of 10^{-4} produced smoother loss curves and more stable gradient updates. Since semantic segmentation requires accurate pixel-level predictions across many parameters, gradual parameter updates allow the network to refine spatial boundaries more effectively. Therefore, 10^{-4} was selected as the optimal learning rate.

Batch Size. Batch size determines the number of samples used to estimate gradients in each optimization step. Smaller batch sizes introduce higher stochasticity in gradient estimates, which can help generalization but may also produce noisy updates. Larger batch sizes provide more stable gradients but may reduce generalization performance and require higher computational resources.

Three batch sizes were tested: 4, 8, and 16. A very small batch size (BS = 4) produced highly noisy gradient estimates, leading to unstable training curves. On the other hand, a larger batch size (BS = 16) resulted in slower improvements in the segmentation score and showed signs of reduced generalization capability.

A batch size of 8 provided the best balance between gradient stability and stochasticity. It produced smoother training curves, efficient GPU utilization, and consistently better segmentation scores across epochs.

Combined Effect. The interaction between learning rate and batch size significantly affects the optimization process. A smaller learning rate combined with a moderate batch size allows the optimizer to make stable yet sufficiently informative updates.

Empirically, the configuration of learning rate 1×10^{-4} and batch size 8 produced the most stable convergence and the highest validation scores. Consequently, this configuration was used for the final model training.

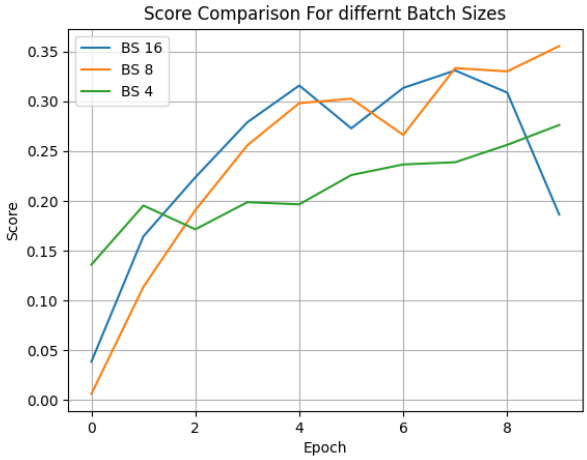


Figure 5: Comparison of segmentation scores across epochs for different batch sizes (BS = 4, 8, 16).

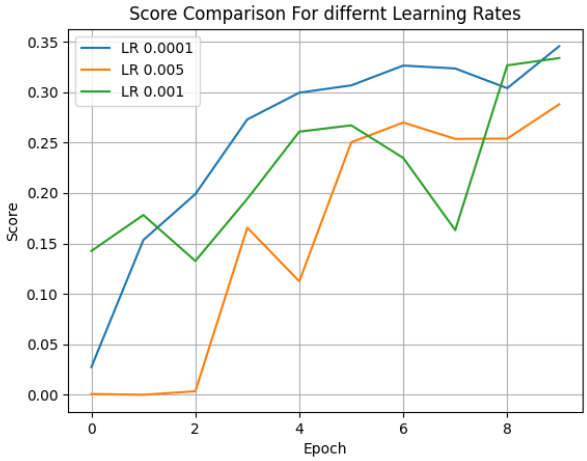


Figure 6: Comparison of segmentation scores across epochs for different learning rates (LR = 10^{-4}, 10^{-3}, 5×10^{-3}).

9.7 Regularization Techniques

Two main regularization strategies were used to improve model generalization.

9.7.1 Batch Normalization

Batch normalization layers were included after each convolution layer. These layers normalize feature distributions during training and help stabilize gradient propagation.

Benefits include faster convergence and improved training stability.

9.7.2 Data Augmentation

To increase dataset diversity and reduce overfitting, several augmentation techniques were applied during training:

- Random horizontal flips
- Random vertical flips
- Random rotations
- Image resizing

These augmentations allow the model to become invariant to orientation changes and improve robustness when encountering new cell images.

9.8 Training Strategy

The network was trained for a maximum of **40 epochs**. Early stopping was used to prevent overfitting by monitoring validation performance. The best-performing model was saved based on the combined validation score computed using IoU and Dice metrics.

During each training iteration, the following steps were performed:

1. Forward propagation through the network
2. Cross-entropy loss computation
3. Backpropagation
4. Parameter updates using the Adam optimizer
5. Validation evaluation using IoU and Dice metrics

9.9 Results

The final segmentation model combines a lightweight U-Net architecture with stable optimization and appropriate evaluation metrics. The chosen hyperparameters provide a balance between computational efficiency and segmentation accuracy.

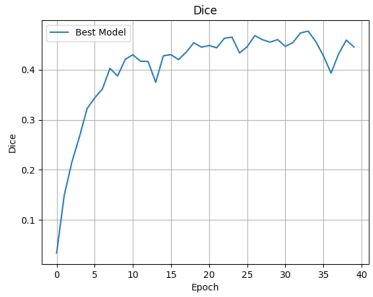
Key experimental settings are summarized below:

- **Architecture:** U-Net encoder–decoder network with skip connections
- **Loss Function:** Cross-Entropy Loss

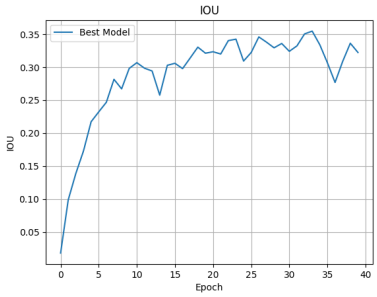
- **Optimizer:** Adam
- **Learning Rate:** 1×10^{-4}
- **Batch Size:** 8
- **Evaluation Metrics:** Dice Score and Intersection over Union (IoU)

During training, multiple performance indicators were monitored to evaluate both learning stability and segmentation quality. These include the Dice score, IoU score, training accuracy, training loss, validation loss, and overall validation score.

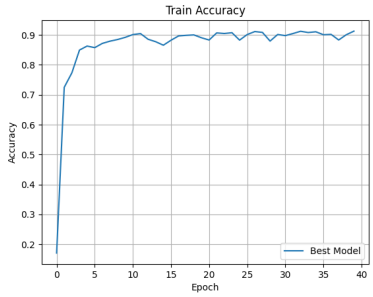
The following figures illustrate the training dynamics across epochs.



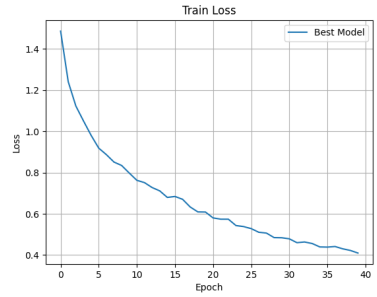
(a) Dice Score



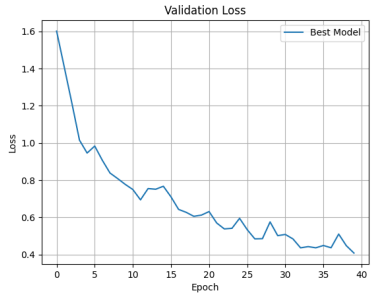
(b) Intersection over Union



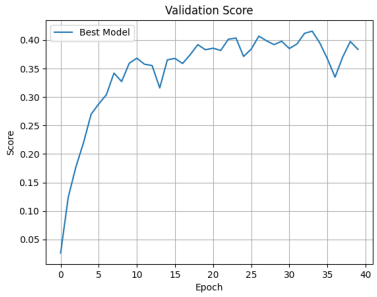
(c) Training Accuracy



(d) Training Loss



(e) Validation Loss



(f) Validation Score

Figure 7: Training and validation metrics recorded during model optimization.

9.10 Conclusion

In this work, a U-Net based convolutional neural network was used to perform multi-class cell segmentation from microscopy images. The encoder, decoder architecture with skip connections helped the model capture both high-level contextual information and fine spatial details necessary for accurate pixel-wise predictions.

Through systematic experimentation, we carefully adjusted the learning rate and batch size to ensure stable optimization and improved segmentation performance. The best configuration used a learning rate of 1×10^{-4} and a batch size of 8, achieving a good balance between convergence stability and generalization ability.

Training and validation curves show consistent learning behavior, with improvements noted in both Dice score and Intersection over Union across epochs. These results suggest that the model successfully learns important representations of cell structures and can produce reliable segmentation masks.

Overall, the approach shows that a lightweight U-Net architecture, combined with suitable optimization strategies and data augmentation techniques, can effectively tackle the cell segmentation problem while remaining computationally efficient.